



PackSwift: AI-Powered Personal Travel Planning Assistant

Khaing Khant Kyaw
6605140002



**AN INDEPENDENT STUDY SUBMITTED IN PARTIAL
SATISFACTION OF THE REQUIREMENTS FOR THE
BACHELOR DEGREE OF INFORMATION TECHNOLOGY
SIAM UNIVERSITY**

2026



PackSwift: AI-Powered Personal Travel Planning Assistant

KHAING KHANT KYAW

This Independent Study Has Been Approved as Meeting Part of the
Requirements for the Bachelor Degree of Information Technology



Advisor

Date/...../.....

Siam University, Bangkok, Thailand

ABSTRACT

Title: PackSwift: AI-Powered Personal Travel Planning Assistant

By: Khaing Khant Kyaw

Degree: Bachelor Degree of Information Technology

Major: IT

Advisor: Dr. Parham Porouhan (Ph.D.)

This study presents the design and methodology of PackSwift, a web-based intelligent travel planning assistant created to reduce the fragmented information-seeking process that commonly precedes a trip. Travelers frequently consult separate tourism sites, weather services, country references, blogs, and discussion forums before they can answer basic planning questions. Repeating this work increases time, cognitive effort, and the possibility that important destination, climate, cultural, or preparation information will be missed.

PackSwift consolidates those decisions into one structured workflow. A user provides a budget, proposed dates, destination preferences, trip purpose, duration, and traveler composition. The system then applies explainable rules to compare destination suitability, summarize historical climate patterns, retrieve destination and attraction information, assemble a practical itinerary, generate cultural and safety notes, and prepare packing lists for men, women, children, and pets. The term AI-powered in this study denotes rule-based reasoning and automated data synthesis; the prototype does not train or deploy a machine-learning model.

The methodology is backend-first and uses Node.js, Express.js, HTML, CSS, Vanilla JavaScript, and MySQL. Open-Meteo supplies historical and climate data, Amadeus supplies destination experiences and points of interest, and REST Countries supplies normalized



country facts. The resulting architecture separates interface, route, service, persistence, and integration responsibilities. The study contributes a technically feasible capstone design for a minimalist personal travel advisor that reduces prompt fatigue and presents traceable recommendations without acting as a reservation service.

Keywords: travel planning assistant, destination recommendation, climate analysis, itinerary generation, packing recommendation, context-aware tourism, rule-based intelligence, Node.js, MySQL



ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my advisor, Dr. Parham Porouhan (Ph.D.), for the valuable guidance, constructive suggestions, and continuous support provided throughout the preparation of this report. The advisor's encouragement and academic direction greatly contributed to the completion and improvement of this study.

I would also like to extend my appreciation to the lecturers and staff of Siam University for their support throughout my studies. Their teaching, professional insight, and academic commitment have played an important role in my learning and development.

Finally, I am deeply grateful to my family, friends, and colleagues for their patience, encouragement, and moral support during the preparation of this project. Their confidence in me has been a constant source of motivation.

Khaing Khant Kyaw



DECLARATION

I, Khaing Khant Kyaw, hereby declare that the work embodied in this report entitled "PackSwift: AI-Powered Personal Travel Planning Assistant" is the result of my own original study and has not been submitted, either in part or in full, for any other degree or academic qualification at any other university or institution.

Khaing Khant Kyaw
28th April 2026



Table of Contents

ABSTRACT.....	3
ACKNOWLEDGEMENT.....	5
DECLARATION.....	6
CHAPTER ONE INTRODUCTION.....	9
1.1 Background of the Study.....	9
1.2 Problem Statement.....	9
1.3 Aim of the Project.....	10
1.4 Objectives of the Study.....	10
1.5 Motivation for the Study.....	11
1.6 Scope of the Study.....	12
1.7 Beneficiaries of the Study.....	13
1.8 Distinctiveness of the Project.....	13
1.9 Limitations of the Study.....	14
1.10 Summary of Chapter One.....	15
CHAPTER TWO RELATED WORKS / LITERATURE REVIEW.....	16
2.1 Introduction.....	16
2.2 Foundations of Travel Recommendation.....	16
2.3 Personalized Destination and Travel Guidance.....	17
2.4 Context-Aware Tourism Systems.....	18
2.5 Intelligent Itinerary Generation.....	19
2.6 Tourism Information and Planning Assistants.....	20
2.7 Research Gap and Positioning.....	20
2.8 Conceptual Foundation of PackSwift.....	20
2.9 Summary of Chapter Two.....	20



CHAPTER THREE METHODOLOGY.....	21
3.1 Introduction.....	21
3.2 Development Approach.....	21
3.3 System Architecture and Design.....	22
3.3.1 Architectural Layers.....	22
3.3.2 End-to-End Data Flow.....	23

3.4 Technology Stack and Tools.....	24
3.4.1 Node.js for Backend Execution.....	24
3.4.2 Express.js for Routing and Middleware.....	24
3.4.3 HTML, CSS, and Vanilla JavaScript.....	24
3.4.4 MySQL for Relational Data Storage.....	24
3.5 Algorithms and Logic of the Planning Engines.....	25
3.5.1 Destination Analysis Engine.....	26
3.5.2 Climate Analysis Engine.....	27
3.5.3 Travel Guide Generator.....	28
3.5.4 Packing Recommendation Engine.....	30
3.5.5 Rule Coordination and Traceability.....	31
3.6 Database Schema and Storage Design.....	32
3.6.1 Core Tables.....	32
3.6.2 Relationship Structure.....	35
3.6.3 Why Relational Storage Is Appropriate.....	36
3.7 Datasets and External Resources.....	37
3.7.1 Open-Meteo.....	37
3.7.2 Amadeus APIs.....	37
3.7.3 REST Countries API.....	37
3.7.4 Data Quality and Resilience.....	38
3.8 Development Planning.....	39
3.8.1 Phase One: Database Design.....	39
3.8.2 Phase Two: Node.js Backend Setup.....	41
3.8.3 Phase Three: API Integration.....	43
3.8.4 Phase Four: Climate Analysis Module.....	45
3.8.5 Phase Five: Destination Intelligence Module.....	47
3.8.6 Phase Six: Packing Recommendation Engine.....	49



3.8.7 Phase Seven: Itinerary Generator.....	51
3.8.8 Phase Eight: Frontend Development.....	53
3.8.9 Phase Nine: Testing and Evaluation.....	54
3.9 Summary of Chapter Three.....	55
REFERENCES.....	56

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Travel planning is an information-intensive activity. A person who suddenly decides to travel may need to compare possible countries, estimate whether the trip is affordable, understand seasonal conditions, identify worthwhile attractions, learn local customs, consider safety concerns, and determine what to prepare. Although digital information is abundant, it is distributed across tourism sites, weather services, country databases, blogs, and travel forums. The traveler must therefore assemble a coherent plan from sources that differ in structure, freshness, terminology, and credibility.

Research on tourism recommender systems has consistently identified information overload and contextual complexity as central design problems. Destination suitability depends on more than popularity: budget, travel period, duration, personal interests, location, and weather can materially change the relevance of a suggestion (Borràs et al., 2014; Gavalas et al., 2014). A useful assistant should consequently transform scattered facts into a sequence of understandable decisions rather than return an



undifferentiated list of links.

PackSwift is proposed as a web-based personal travel planning assistant. Its organizing idea is simple: when a user decides to travel, the system should help plan the trip in one place. It accepts structured inputs, retrieves relevant external information, applies transparent rules, and presents destination guidance, climate expectations, attractions, an itinerary, preparation advice, and packing lists through a minimalist interface.

1.2 Problem Statement

The central problem is that travel planning information is fragmented across blogs, weather websites, tourism websites, and travel forums. Users repeatedly change context, re-enter dates and destinations, reconcile conflicting statements, and decide which information should influence the final plan. This fragmented workflow consumes time and increases cognitive load before the traveler has produced an actionable result.

Fragmentation also creates consistency and trust problems. A popular attraction may be unsuitable during a rainy period; a destination that appears affordable may conflict with the user's duration or expected daily budget; and general advice may omit the needs of children or pets. Safety and cultural information can become detached from the itinerary, while packing advice may be generic because it is not connected to expected temperature, rainfall, purpose, or trip length.

General-purpose conversational systems can answer travel questions, but they frequently require users to formulate several prompts, restate constraints, and evaluate differently structured answers. In this study, prompt fatigue means the repeated effort required to decide what to ask, supply the same context again, and convert open-ended responses into a usable plan. PackSwift addresses this operational problem by collecting essential constraints once and leading the user through a predefined planning sequence.

The project therefore addresses a workflow gap rather than a lack of online information. Travelers need a system that consolidates reliable data, explains why recommendations were produced, preserves source freshness, and turns destination research into a structured plan. The proposed solution is deliberately advisory: it supports preparation and decision making and does not attempt to reserve transport or accommodation.

1.3 Aim of the Project

The aim of this project is to design and develop an AI-powered personal travel planning assistant that consolidates destination, climate, attraction, cultural, safety, itinerary, preparation, and packing guidance into one explainable web-based workflow using rule-based intelligence and approved external APIs.



1.4 Objectives of the Study

1. To recommend suitable destinations by evaluating the user's budget, dates, duration, destination constraints, and stated interests.
2. To analyze likely temperature and rainfall conditions for the intended travel period using historical and climate statistics.
3. To generate intelligent packing checklists for men, women, children, and pets using destination, climate, duration, and trip purpose.
4. To provide concise destination intelligence, including country facts, languages, currency, regional context, and preparation notes.

1. To suggest relevant attractions and places of interest using preference, location, popularity, available time, and climate suitability rules.
2. To generate a practical day-by-day itinerary that groups compatible attractions and respects duration, opening information when available, and reasonable daily intensity.
3. To present cultural guidance and time-stamped safety notes while clearly communicating the limits and source status of such information.
4. To present the complete plan through a simple and minimalist interface implemented with HTML, CSS, and Vanilla JavaScript.

Together, these objectives define an integrated planning assistant rather than a collection of disconnected utilities. Each output must be traceable to a user input, an external source, or a documented rule so that the user can understand and revise the result.

1.5 Motivation for the Study

The study is motivated by efficiency, confidence, and accessibility. First, consolidating core planning tasks reduces repeated searches and lets the traveler move from a vague intention to an organized plan. Second, explainable recommendations can increase confidence because the system states the factors behind a destination, packing item, or itinerary decision. Third, a structured interface supports users who may not know which questions to ask or how to judge large volumes of tourism information.

The project is also motivated by technical feasibility. Rule-based reasoning can provide meaningful personalization without an opaque predictive model. Historical climate statistics can be calculated from external data, points of interest can be normalized into a consistent format, and MySQL can preserve both structured results and source metadata. This approach is suitable for a



university capstone because the reasoning is inspectable, testable, and achievable with the specified JavaScript stack.

A minimalist interface further supports the motivation. PackSwift should reveal information progressively: essential trip constraints first, destination analysis second, then climate, attractions, itinerary, preparation, and packing. WCAG 2.2 principles guide semantic structure, keyboard access, status messages, visible focus, and understandable feedback so that simplicity does not come at the expense of accessibility (World Wide Web Consortium, 2024).

1.6 Scope of the Study

The scope covers the design and prototype implementation of a browser-based travel planning assistant. The user can create a trip session, enter a total or daily budget, specify dates and duration, identify a preferred or undecided destination, select interests and trip purpose, and describe the traveler group. The server validates these inputs and coordinates the planning modules.

- destination comparison and suitability scoring based on budget, dates, duration, region, and preferences;
- historical temperature and rainfall analysis for the relevant seasonal window;
- country intelligence covering currency, languages, population, region, capital, and time-zone context;
- retrieval and ranking of attractions and points of interest;
- generation of a day-by-day itinerary with morning, afternoon, and evening suggestions;
- cultural notes, preparation guidance, and locally maintained safety summaries;
- packing lists for men, women, children, and pets, with quantities adjusted for duration and purpose;
- storage of trip sessions, derived insights, source snapshots, generated lists, and revision history;
- a minimalist responsive interface created with semantic HTML, CSS, and Vanilla JavaScript.

The server is implemented with Node.js and Express.js, and MySQL is used for relational persistence. The permitted external integrations are Open-Meteo for historical and climate data, Amadeus for destination experiences and points of interest, and REST Countries for country facts. The first version uses rule tables, thresholds, weighted criteria, and deterministic sequencing.



It does not contain a trained predictive model; the browser interface also requires no separate device-specific client.

The scope excludes transport and accommodation reservation, price guarantees, visa determination, emergency response, and authoritative risk certification. Estimated affordability is an advisory comparison based on configured cost bands and user input; it is not a quotation. Safety notes are displayed with source and review timestamps and must encourage verification through the appropriate public authority before departure.

1.7 Beneficiaries of the Study

The primary beneficiaries are travelers who need a clear starting point for trip preparation. First-time travelers can receive an ordered plan without knowing every question in advance. Budget-conscious users can compare destinations against an explicit limit. Solo travelers and couples can identify relevant attractions, while families can receive child-aware preparation and pacing. Travelers accompanied by pets can receive separate equipment, document, comfort, and feeding reminders.

Students and developers also benefit from the project as a transparent case study in service-oriented web design, external API normalization, relational modeling, rule-based reasoning, and explainable recommendation. Tourism educators and capstone evaluators can inspect the connection between objectives, algorithms, database records, and evaluation criteria. Because the interface uses standard browser technologies, it can also support users with varied levels of digital experience.

1.8 Distinctiveness of the Project

PackSwift is distinctive because it is a structured AI-powered travel assistant that removes prompt fatigue and consolidates travel planning into a single platform. It does not wait for users to invent a sequence of questions. Instead, the interface captures the planning context once, invokes purpose-built engines, and presents outputs in a stable order. The result resembles the disciplined guidance of a personal travel advisor while remaining explainable and technically modest.

- one structured intake replaces repeated context entry;



- destination, climate, attraction, itinerary, cultural, safety, and packing outputs share the same trip session;
- recommendations include human-readable reasons and source timestamps;
- climate expectations are separated from certainty claims;
- traveler categories are adjustable and do not force stereotyped assumptions;
- the interface emphasizes decisions and preparation rather than open-ended prompting.

This combination positions PackSwift between a static tourism information site and an unrestricted conversational assistant. It provides personalization without requiring the user to master prompting and provides automation without concealing the rule logic that generated each suggestion.

1.9 Limitations of the Study

The project has several practical limitations. First, outputs depend on external API availability, coverage, licensing conditions, response formats, and rate limits. A temporary failure may require cached data or a clear unavailable-state message. Amadeus attraction coverage may vary by destination, and REST Countries supplies country-level facts rather than city-level cultural interpretation.

Second, historical and climate data describe patterns rather than guaranteed conditions. Averages can hide unusual heat, storms, or rainfall. The interface must therefore label the observation period, sample size, source, and uncertainty, and it must avoid presenting a climate estimate as a precise future forecast.

Third, rule-based intelligence is transparent but limited. It cannot infer every preference, mobility constraint, religious practice, medical need, or personal packing habit. Thresholds and weights require validation, and users must be able to edit generated results. Men, women, children, and pet lists are configurable profiles; they are not universal statements about what every traveler in a category requires.

Fourth, destination affordability is an estimate. The selected APIs do not provide a complete real-time cost model for every country, and exchange conditions can change. The first prototype therefore uses maintained cost bands, configurable assumptions, and explanatory ranges rather than exact promises.

Fifth, cultural and safety guidance can become outdated or oversimplified. The system stores review dates, distinguishes general



etiquette from formal requirements, and presents warnings as supplemental information. Users remain responsible for verifying visas, entry conditions, health requirements, and current public advisories through authoritative sources.

Finally, the first version prioritizes a web prototype and core planning flow. Offline access, collaborative trip editing, multilingual authoring, real-time disruption response, advanced route optimization, and native device features are outside the present scope. These limitations define a realistic boundary for evaluation and identify directions for later work.

1.10 Summary of Chapter One

This chapter established the background, problem, aim, objectives, motivation, scope, beneficiaries, distinctiveness, and limitations of PackSwift. The study responds to the fragmentation of travel planning information across tourism sites, climate resources, country references, blogs, and forums. Its central contribution is a structured planning workflow that reduces repeated prompting and converts shared trip context into destination, climate, attraction, itinerary, cultural, safety, preparation, and packing guidance.

The proposed system is intentionally advisory, explainable, and feasible. It uses rule-based intelligence and API integration within a Node.js, Express.js, HTML, CSS, Vanilla JavaScript, and MySQL architecture. It does not rely on a trained predictive model and does not attempt to complete reservations. Chapter Two reviews the academic foundations for travel recommendation, context-aware tourism, personalized guidance, intelligent itinerary generation, and tourism information systems, then identifies the research gap addressed by the project.



CHAPTER TWO

RELATED WORKS / LITERATURE REVIEW

2.1 Introduction

This chapter reviews research relevant to PackSwift as a personal travel planning assistant. The discussion focuses on travel recommendation, context-aware tourism, personalized guidance, itinerary generation, and tourism information systems. The reviewed studies demonstrate that useful tourism support depends on connecting user preferences with contextual constraints and converting large information spaces into manageable decisions.

2.2 Foundations of Travel Recommendation

Felfernig et al. (2006) surveyed collaborative, content-based, and knowledge-based recommendation technologies in travel and tourism. Their analysis is important because travel decisions involve composite constraints and relatively infrequent choices, conditions under which explicit knowledge and user requirements can be more useful than behavior history alone. PackSwift adopts this insight



through structured questions and rules rather than relying on prior usage data.

Kabassi (2010) reviewed personalization methods for tourists and emphasized the need to model user characteristics before presenting suggestions. The study also highlighted the difficulty created by a large number of online alternatives. This supports PackSwift's decision to collect budget, dates, duration, interests, purpose, and traveler composition as explicit planning variables.

Borràs et al. (2014) provided a comprehensive survey of intelligent tourism recommender systems, covering destination selection, attraction recommendation, multi-day planning, interfaces, and artificial-intelligence methods. The authors showed that contextual factors and automated planning extend recommendation beyond static ranking. Their taxonomy supplies a broad conceptual foundation for the integrated modules proposed in this study.

2.3 Personalized Destination and Travel Guidance

Liu et al. (2011) modeled personalized travel package recommendation by combining tourist interests with destination and seasonal characteristics. Although the present system does not assemble supplier packages, the study demonstrates that destination relevance should be judged through multiple attributes rather than popularity alone. PackSwift similarly treats season, preference fit, duration, and affordability as separate explainable criteria.

Ravi and Vairavasundaram (2016) developed a location-based recommendation approach for groups of users. Their work is relevant because travel preferences may represent more than one person and because group satisfaction cannot be reduced to a single individual's rating. PackSwift does not implement collaborative prediction, but its traveler composition and adjustable pacing rules acknowledge that family and companion needs affect destination, attraction, itinerary, and packing decisions.

Gavalas et al. (2014) reviewed mobile tourism recommender systems and classified common functions such as attraction suggestion, routes, tours, and multi-day itinerary planning. The review emphasized personal, social, and environmental context. Even though PackSwift is a web application rather than a native mobile system, it uses the same principle of contextual integration and presents the result through a responsive browser interface.

Gretzel et al. (2015) described smart tourism as the transformation of data into relevant information, decision support, and improved experiences. This perspective is directly applicable to PackSwift: the value of the system does not come from merely retrieving data, but from normalizing, interpreting, linking, and explaining it within a specific travel session.



2.4 Context-Aware Tourism Systems

Noguera et al. (2012) combined a hybrid recommendation engine with a location-sensitive tourism interface. Their system demonstrated that physical context and usable presentation can improve the relevance of points-of-interest guidance. PackSwift extends the contextual view before travel by considering intended dates, destination, budget, duration, and purpose, while retaining the principle that interface design and recommendation logic must be evaluated together.

Braunhofer et al. (2013) designed STS, a weather-aware tourism recommender, and reported improved choice satisfaction and perceived recommendation quality in a live user study. The study is particularly significant for PackSwift because climate is not treated as decorative information. Temperature and rainfall categories influence attraction suitability, itinerary notes, preparation guidance, and packing recommendations.

Namahoot et al. (2015) integrated temporal ontology and Naive Bayes classification in a context-aware tourism recommender. Although PackSwift excludes statistical learning, the study reinforces the importance of temporal representation. The proposed system uses deterministic seasonal windows, date overlap, climate summaries, and rule priorities so that temporal effects remain explicit and testable.

Abbasi-Moud et al. (2022) proposed a fuzzy ontology-based tourism recommendation system that integrates semantic and contextual information. Its contribution shows how richer context can improve relevance, but it also increases methodological complexity. PackSwift adopts a simpler rule matrix and weighted scoring method suited to a capstone prototype, while preserving semantic labels and explanation records for later extension.



2.5 Intelligent Itinerary Generation

Souffriau et al. (2008) formulated personalized tourist trip design as a constrained selection and routing problem. Their method sought to maximize the value of visited locations within an available time or distance budget. This work establishes that itinerary generation is not simply a list-sorting task; it requires explicit constraints and a meaningful measure of attraction relevance.

Kurashima et al. (2010) generated travel route recommendations from geotagged photographs. The study demonstrated that collective traces can reveal meaningful sequences of visited places. PackSwift does not mine social-media behavior, but it uses the underlying lesson that ordering should reflect geographic and experiential coherence rather than independent attraction scores alone.

De Choudhury et al. (2010) automatically constructed intra-city itineraries from geo-temporal traces left by travelers. Their work shows the value of time-aware evidence when identifying plausible visiting patterns. In the present system, itinerary blocks use explicit duration, time-of-day suitability, approximate visit length, and geographic grouping, with source limitations shown to the user.

Vansteenwegen et al. (2011) introduced the City Trip Planner, a web-based expert system that generated personalized multi-day routes while considering interests, opening hours, visit duration, and meal breaks. This is the closest methodological precedent for PackSwift's itinerary component. However, PackSwift positions itinerary generation within a wider pre-travel workflow that also includes destination comparison, climate analysis, cultural guidance, preparation, and packing.



2.6 Tourism Information and Planning Assistants

The literature collectively shows that tourism information systems become more valuable when they reduce overload, represent context, and transform data into a decision sequence. Surveys by Borràs et al. (2014) and Gavalas et al. (2014) identify destination, attraction, route, and itinerary functions, while Gretzel et al. (2015) explains the broader role of connected information in smart tourism. Nevertheless, many research prototypes concentrate on one stage, one device context, or one recommendation target.

2.7 Research Gap and Positioning of the Proposed System

A gap remains between specialized recommenders and a complete, low-friction planning assistant. Destination studies may not produce a preparation plan; weather-aware systems may focus on activities at the destination; itinerary systems may assume the destination has already been selected; and general assistants may require repeated, open-ended prompting. There is therefore room for a structured web system that captures context once and coordinates several transparent planning engines.

2.8 Conceptual Foundation of PackSwift

PackSwift combines four principles derived from the literature: explicit user modeling, context-aware filtering, constrained itinerary construction, and integrated tourism information. Its distinctive contribution is orchestration. A single trip session links budget and preferences to destination suitability; dates to climate statistics; climate and interests to attractions; attraction constraints to an itinerary; and the full context to preparation and packing. Explanations and source timestamps are stored with the generated outputs.

2.9 Summary of Chapter Two



This chapter reviewed fifteen relevant studies and established the academic basis for personalized, context-aware, and itinerary-oriented tourism support. The findings justify a rule-based architecture in which context is explicit, outputs are linked, and users can revise the plan. Chapter Three converts these principles into a backend-first methodology, database design, service architecture, algorithms, integration strategy, and nine-phase implementation roadmap.

CHAPTER THREE

METHODOLOGY

3.1 Introduction

This chapter explains how PackSwift will be designed, implemented, and evaluated as an AI-powered personal travel planning assistant. The methodology is practical, backend-first, and traceability-oriented. It translates the study objectives into architectural components, deterministic planning rules, relational records, external API contracts, and testable deliverables.

3.2 Development Approach

The project follows an iterative backend-first approach. Database relationships, validation rules, service boundaries, and integration adapters are established before interface screens are connected. This sequence is appropriate because every visible result depends on a stable trip session and on consistent coordination among destination, climate, attraction, itinerary, cultural, safety, and packing data.



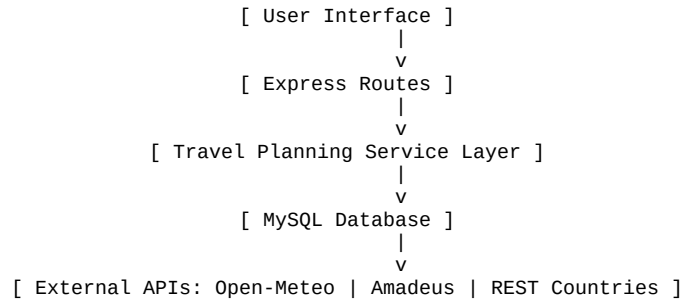
Development proceeds through short increments. Each increment defines an input contract, implements a service function, persists its output, exposes a route, and verifies both expected and failure behavior. The interface is attached only after the corresponding service returns a documented response shape. This reduces rework and makes it possible to test the reasoning without relying on visual interaction.

1. define the nine required relational tables and their constraints;
2. initialize the Node.js project and Express.js service structure;
3. create provider adapters and normalized response objects;
4. implement climate, destination, guide, packing, and itinerary rules;
5. expose versioned REST endpoints with validation and error handling;
6. connect the HTML, CSS, and Vanilla JavaScript interface;
7. evaluate correctness, usability, explainability, resilience, and performance.

3.3 System Architecture and Design

PackSwift uses a layered web architecture. Each layer has a narrow responsibility and communicates through explicit data structures. The required functional progression is shown in Figure 3.1. The MySQL layer stores both user-generated context and normalized snapshots; the external API layer supplies source data that is requested and transformed by the travel planning services.

Figure 3.1 Overall PackSwift Architecture



The vertical representation describes the order in which concerns are introduced to the user and persisted. At runtime, the service layer



communicates directly with both MySQL repositories and provider adapters; the database does not call external services. This distinction prevents an incorrect dependency between persistence and network integration.

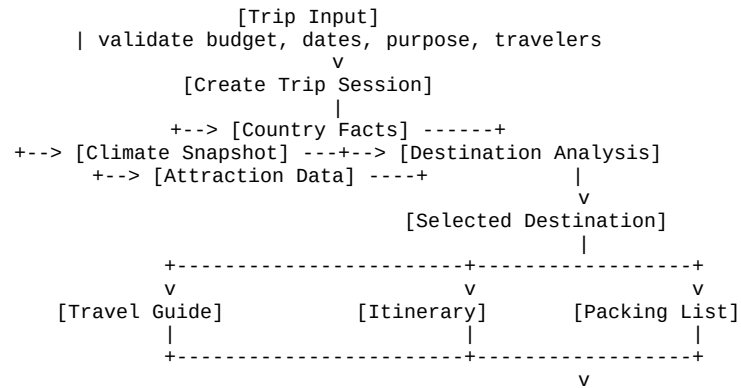
3.3.1 Architectural Layers

- User Interface: semantic pages and forms that collect trip context and render planning results.
- Express Routes: endpoint matching, request validation, session checks, response formatting, and error delegation.
- Travel Planning Service Layer: orchestration, scoring, climate aggregation, list generation, itinerary sequencing, and explanation creation.
- MySQL Database: durable trip context, normalized source snapshots, generated outputs, and timestamps.
- External APIs: approved providers for climate, destination experiences, points of interest, and country facts.

3.3.2 End-to-End Data Flow

A planning request moves through a controlled sequence so that later outputs reuse earlier context instead of requesting it again. The sequence also allows the interface to report partial progress and recover from an unavailable provider.

Figure 3.2 Planning Data Flow



[Unified Plan View]

1. The browser submits one validated planning context to the server.
2. The server creates a draft trip session and assigns a unique identifier.
3. Provider adapters retrieve only the fields required for candidate destinations and save source timestamps.
4. The destination engine scores each candidate and records a reason for every criterion.
5. After selection, the service layer generates destination guidance, itinerary blocks, and packing items.
6. The interface retrieves the unified plan and allows the user to edit or regenerate individual sections.

All generated records carry a trip identifier, generation time, rule version, and source status. This supports auditability and prevents outputs from different sessions from being combined accidentally.

3.4 Technology Stack and Tools

3.4.1 Node.js for Backend Execution

Node.js provides the JavaScript runtime for the server. Its asynchronous event-driven model is suitable for coordinating several network requests without blocking the entire application while a provider responds (Node.js, n.d.). One language can therefore be used across the server and browser while responsibilities remain separated by modules.

3.4.2 Express.js for Routing and Middleware

Express.js defines versioned endpoints and composes middleware for JSON parsing, validation, authentication checks, request logging, and centralized error handling. Route handlers remain thin: they validate input, call a service, and format the result. Express documentation describes routes as endpoint responses and applications as middleware sequences, which supports this layered structure (Express.js, n.d.-a, n.d.-b).

3.4.3 HTML, CSS, and Vanilla JavaScript for Frontend Delivery

Semantic HTML provides accessible forms and content structure, CSS provides a restrained responsive layout, and Vanilla JavaScript uses the browser Fetch API to communicate with the server. The interface uses progressive disclosure: one planning stage is emphasized at a time, while completed stages remain available for revision. No frontend framework is required.

3.4.4 MySQL for Relational Data Storage



MySQL stores strongly related entities such as users, trip sessions, destination insights, climate snapshots, attractions, itineraries, packing lists, cultural guides, and travel advisories. Foreign keys preserve ownership and deletion rules, indexes support common lookups, and JSON columns can retain selected provider fragments without replacing queryable relational fields. MySQL supports numeric, temporal, text, spatial, and JSON data types suitable for this design (Oracle, n.d.).

Table 3.1 Technology Responsibilities

Layer	Selected Technology	Primary Responsibility
Server runtime	Node.js	Asynchronous application execution
HTTP layer	Express.js	Routes, middleware, validation, errors
Browser layer	HTML, CSS, Vanilla JavaScript	Input, navigation, and result presentation
Persistence	MySQL	Relationships, source snapshots, generated outputs

3.5 Algorithms and Logic of the Planning Engines

The intelligence of PackSwift is implemented through deterministic rules, weighted criteria, thresholds, and ordered transformations. This method is appropriate because the prototype must be explainable and must not depend on a trained predictive model. Every recommendation is accompanied by a reason code, plain-language explanation, and the input values that activated the rule.

Table 3.2 Planning Engine Contracts

Engine	Principal Inputs	Principal Outputs
Destination Analysis	Budget, dates, duration, region, interests, purpose	Suitability score, affordability band, rationale
Climate Analysis	Coordinates, date window, historical years	Temperature range, rainfall pattern, confidence note
Travel Guide	Selected destination, climate, interests, duration	Attractions, cultural notes, safety notes, preparation
Packing Recommendation	Climate category, duration, purpose, traveler profiles	Grouped items, quantities, reasons, priority
Itinerary Generation	Ranked attractions, duration, visit times, location	Day, sequence, time block, notes

The engines share a canonical trip context object. It contains the trip session identifier, start and end dates, duration, budget amount and currency, candidate or selected destination, purpose, interest tags, pace preference, mobility notes, and traveler counts. Provider data is normalized before it enters an engine so that external field changes do not spread through the application.



Rule evaluation follows three priorities. A safety or feasibility rule may exclude a result; a relevance rule may increase or decrease its score; and a presentation rule determines grouping or explanation. When two rules conflict, exclusion rules take precedence, followed by higher-priority relevance rules. A stable final sort uses score, confidence, and name so that identical inputs produce identical outputs.

$$\text{generated output} = \text{normalized inputs} + \text{applicable rules} + \text{source evidence} + \text{explanation}$$

3.5.1 Destination Analysis Engine

The Destination Analysis Engine compares candidate countries or cities against the user's constraints. When a destination is already known, the engine evaluates suitability and preparation implications rather than replacing the user's choice. When the destination is undecided, the engine ranks a curated candidate set supported by available provider data.

Table 3.3 Destination Suitability Criteria

Criterion	Measure	Weight
Budget fit	Estimated trip band compared with stated limit	35%
Preference fit	Matched interest and purpose tags	25%
Climate fit	Expected conditions compared with stated tolerance	20%
Duration fit	Travel period compared with useful visit length	10%
Seasonal suitability	Seasonal access and activity relevance	10%

$$\text{destination score} = 0.35B + 0.25P + 0.20C + 0.10D + 0.10S$$

B, P, C, D, and S are normalized values from 0 to 100. A candidate is excluded only by a documented hard constraint, such as a user-specified prohibited region or a missing minimum data set. Affordability uses ranges rather than precise costs. The estimate records its currency, configured assumptions, and last review date.



1. Validate the budget, currency, dates, and duration.
2. Build or retrieve a supported candidate destination set.
3. Attach country facts and matching climate statistics.
4. Calculate criterion scores and preserve the component values.
5. Apply exclusions and reduce confidence when source coverage is incomplete.
6. Return the highest-ranked candidates with strengths, cautions, and assumptions.

For example, a candidate may receive a high preference score for beaches and food, a moderate budget score, and a low climate score because the intended period has a high rainy-day ratio. The interface shows those components rather than presenting only a single opaque number.

3.5.2 Climate Analysis Engine

The Climate Analysis Engine estimates likely conditions for the planned travel period from historical and climate data. It does not claim to know the exact future weather. For a selected destination, the system identifies coordinates and constructs a seasonal comparison window around the intended month and day. It then retrieves comparable observations across a configured multi-year period and calculates descriptive statistics.

- daily maximum, minimum, and mean temperature;
- mean and median precipitation;
- rainy-day ratio using a documented precipitation threshold;
- frequency of unusually hot, cold, or wet days;
- interannual spread and number of usable observations;
- a plain-language seasonal summary and uncertainty statement.

mean temperature = sum(daily mean temperatures) / number of observed days

rainy-day ratio = days above precipitation threshold / observed days

Table 3.4 Climate Classification Examples



Derived Category	Example Rule	Planning Effect
Cool	Mean daytime temperature below 16 C	Warm layers; indoor alternatives
Hot	Mean daytime temperature at or above 30 C	Hydration; sun protection; slower pace
Frequently wet	Rainy-day ratio at or above 0.40	Rain layer; covered attractions; flexible sequence
Variable	Large temperature or rainfall spread	Layering; explicit uncertainty note

The engine stores the provider, requested coordinates, date range, variables, calculation period, computed values, and retrieval time in `weather_snapshots`. If the data set is incomplete, the result is marked low confidence and the system avoids strong preparation claims. Historical statistics support seasonal expectations; they are not substitutes for checking current conditions near departure.

3.5.3 Travel Guide Generator

The Travel Guide Generator assembles destination information into four coordinated outputs: recommended attractions, cultural notes, safety information, and a preparation checklist. It receives the selected destination, country facts, ranked points of interest, climate categories, trip purpose, duration, and traveler composition. Each output is concise, categorized, and linked to its source or rule.

Table 3.5 Travel Guide Outputs

Output	Generation Method	Stored Evidence
Attractions	Preference and suitability scoring	Provider ID, category, coordinates, score
Cultural notes	Curated guidance filtered by country and topic	Category, text, review date
Safety information	Curated, time-stamped advisory summaries	Severity, source label, review date
Preparation checklist	Rule combination from climate and trip context	Task, priority, reason code

Attraction scoring uses preference match, popularity when available, climate suitability, approximate distance from the daily cluster, and diversity. The engine avoids returning many near-duplicate places from one category unless the user explicitly emphasizes that interest. It also records missing fields rather than inventing an opening time, visit duration, or accessibility feature.

$$attraction\ score = 0.40I + 0.20R + 0.20W + 0.10G + 0.10V$$

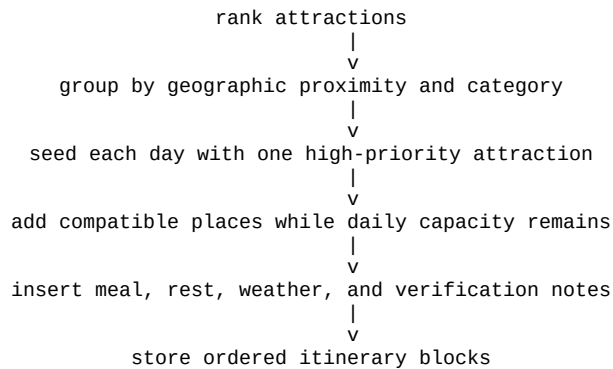


I is interest fit, R is provider relevance or rating, W is climate suitability, G is geographic fit, and V is diversity contribution. A closed or unavailable attraction can be excluded when reliable status information exists. Otherwise the system uses neutral wording and instructs the traveler to verify current operating details.

Cultural guidance is framed respectfully and avoids presenting a population as uniform. Notes use language such as common practice, formal requirement, or suggested courtesy. Users can view the last review date and edit the checklist to reflect personal beliefs, accessibility needs, health needs, and local advice.

The itinerary function transforms the ranked attraction set into a feasible day-by-day sequence. It first reserves the number of planning days, then assigns high-priority attractions to day clusters. Remaining places are added only when approximate visit time, travel allowance, and a rest buffer fit the daily capacity. Morning, afternoon, and evening labels are used instead of false minute-level precision when reliable timing data is unavailable.

Figure 3.3 Itinerary Construction Logic



- Indoor places receive a positive adjustment during frequently wet periods.
- Outdoor places remain available but receive a weather-dependent caution.



- Family pacing reduces the number of major places assigned to one day.
- A high-intensity preference permits more blocks but still preserves a rest buffer.
- The same attraction is not assigned twice unless the user deliberately repeats it.
- Unverified hours are displayed as a check-before-visit task.

Safety notes are not generated from popularity or climate data. They come from locally curated records in travel_advisories and are filtered by destination, date validity, and severity. The interface distinguishes informational, caution, and important notices, displays a review timestamp, and reminds the user to consult the appropriate public authority for current conditions.

Preparation tasks may include travel document verification, local connectivity, currency access, climate readiness, medication planning, child documentation, or pet requirements. The task wording remains advisory and does not claim legal or medical authority.

3.5.4 Packing Recommendation Engine

The Packing Recommendation Engine generates editable lists for men, women, children, and pets. It begins with universal travel essentials, then applies destination, climate, duration, trip purpose, cultural guidance, traveler profile, and pet rules. Categories are user-selected organizational views; the engine does not assume that every person within a category has the same needs.

Table 3.6 Packing Rule Signals

Rule Signal	Example Condition	Example Result
Temperature	Cool or variable conditions	Layering garments and a light outer layer
Rainfall	Frequently wet period	Compact rain layer and water-resistant protection
Duration	More than seven days	Quantity increase or laundry reminder
Purpose	Business or formal event	Optional formal clothing and document organizer
Children	Child traveler count above zero	Age-adjusted comfort and identification items
Pets	Pet traveler count above zero	Carrier, feeding, comfort, and record reminders

Every generated item contains a traveler group, category, name, suggested quantity, priority, reason code, explanation, completion state, and editable flag. Quantities are calculated with bounded formulas. For example, a daily clothing item may use the smaller of trip



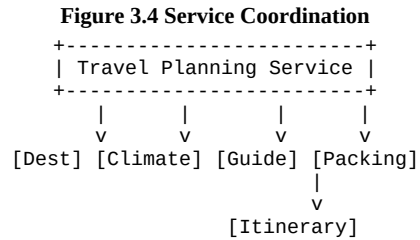
duration and a configured maximum, then add a laundry reminder for longer stays. Essential documents remain quantity one regardless of duration.

$$\text{suggested quantity} = \min(\text{duration-based quantity}, \text{configured maximum}) + \text{context adjustment}$$

- remove duplicates created by overlapping rules;
- place universal items before optional profile-specific items;
- separate carry-on, checked-bag, and destination-use notes only when relevant;
- display the reason for climate-sensitive and purpose-sensitive suggestions;
- allow the traveler to add, delete, rename, complete, or restore items.

3.5.5 Rule Coordination and Traceability

The engines are coordinated by a Travel Planning Service rather than calling one another directly. The coordinator retrieves a canonical trip context, requests required source snapshots, invokes each engine in dependency order, and commits generated records within controlled database operations. This keeps module boundaries clear and allows one output to be regenerated without rebuilding the entire plan.



All modules read normalized context and return:
data + reason codes + source status + rule version

Table 3.7 Explanation and Provenance Fields



Trace Field	Purpose
rule_code	Identifies the deterministic rule that contributed to an output
rule_version	Allows results to be reproduced after rule updates
source_provider	Identifies Open-Meteo, Amadeus, REST Countries, or curated data
source_retrieved_at	Shows when external or curated information was last obtained
confidence_level	Communicates complete, partial, or low-confidence evidence
explanation_text	Provides a plain-language reason visible to the traveler

A generation request uses an idempotency key derived from the trip session, requested module, and revision. Repeating the same request does not create uncontrolled duplicates. When inputs change, the affected records receive a new revision while the trip retains its identity. Logs contain technical identifiers and failure categories but avoid unnecessary personal details.

3.6 Database Schema and Storage Design

The MySQL schema contains nine core tables. The design removes unrelated commercial records and centers persistence on a user's travel planning session. Relational columns store identifiers, dates, scores, status values, and common query fields. JSON is reserved for bounded provider fragments or structured explanations that do not justify additional entities in the capstone scope.

3.6.1 Core Tables

Table 3.8 Core Database Tables



Table	Purpose	Primary Relationship
users	Account identity and preferences	One user has many trip sessions
trip_sessions	Planning input and lifecycle state	Parent of generated trip data
destination_insights	Candidate and selected destination analysis	Many insights per trip session
weather_snapshots	Source values and derived climate statistics	Many snapshots per trip and destination
attractions	Normalized points of interest	Many attractions per destination insight
itineraries	Ordered day and time blocks	Many blocks per trip session
packing_lists	Editable packing items by traveler group	Many items per trip session
cultural_guides	Curated destination guidance	Many notes per trip or destination
travel_advisories	Time-stamped safety summaries	Many notices per trip or destination

The `users` table stores `user_id` as the primary key, `name`, `email`, `password_hash`, `preferred_currency`, `default_budget_style`, `accessibility_notes`, `created_at`, and `updated_at`. Email is unique, passwords are never stored in readable form, and free-text notes are optional. A future privacy function can export or remove the user's planning data by following foreign-key ownership.

The `trip_sessions` table stores `trip_session_id`, `user_id`, `status`, `start_date`, `end_date`, `duration_days`, `budget_amount`, `budget_currency`, `destination_mode`, `selected_country_code`, `selected_city_code`, `trip_purpose`, `pace_level`, `adult_men_count`, `adult_women_count`, `child_count`, `pet_count`, `preference_tags`, `mobility_notes`, `current_revision`, `created_at`, and `updated_at`. CHECK constraints or service validation ensure non-negative counts, valid dates, and a positive duration.

The `destination_insights` table stores `destination_insight_id`, `trip_session_id`, `country_code`, `city_code`, `destination_name`, `estimated_cost_min`, `estimated_cost_max`, `budget_fit_score`, `preference_fit_score`, `climate_fit_score`, `duration_fit_score`, `seasonal_fit_score`, `total_score`, `confidence_level`, `explanation_json`, `selected_flag`, `source_retrieved_at`, and `created_at`. A unique composite key prevents the same destination and revision from being inserted twice.

Table 3.9 Account and Destination Indexes



Table	Important Indexes	Reason
<code>users</code>	UNIQUE(email)	Account lookup and duplicate prevention
<code>trip_sessions</code>	INDEX(user_id, updated_at)	Recent trips for one user
<code>destination_insights</code>	INDEX(trip_session_id, total_score)	Ranked candidates within a trip
<code>destination_insights</code>	UNIQUE(trip_session_id, city_code, revision)	Idempotent generation

Preference tags are stored as a validated JSON array because the set is small, controlled, and primarily read with the session. Frequently filtered destination fields remain relational. Scores use DECIMAL columns to avoid inconsistent rounding, while currency values use DECIMAL and an ISO currency code.

The `weather_snapshots` table stores `weather_snapshot_id`, `trip_session_id`, `destination_insight_id`, `provider`, `latitude`, `longitude`, `comparison_start`, `comparison_end`, `years_analyzed`, `mean_temp_min`, `mean_temp_max`, `mean_temp`, `mean_precipitation`, `rainy_day_ratio`, `hot_day_ratio`, `cool_day_ratio`, `variability_level`, `confidence_level`, `source_payload_json`, `retrieved_at`, and `created_at`. Derived values remain separate from the bounded raw fragment so calculations can be queried and audited.

The `attractions` table stores `attraction_id`, `trip_session_id`, `destination_insight_id`, `provider_place_id`, `name`, `category`, `latitude`, `longitude`, `provider_relevance`, `preference_score`, `climate_suitability_score`, `geographic_cluster`, `estimated_visit_minutes`, `opening_note`, `source_url`, `source_retrieved_at`, and `created_at`. A provider identifier and destination key prevent duplicates.

The `itineraries` table stores `itinerary_id`, `trip_session_id`, `attraction_id` when applicable, `day_number`, `sequence_number`, `time_block`, `title`, `description`, `estimated_start`, `estimated_end`, `transport_note`, `climate_note`, `verification_required`, `reason_code`, `revision`, `created_at`, and `updated_at`. Each row is one ordered plan block, which avoids a hidden list inside a single large document field.

The `packing_lists` table stores `packing_item_id`, `trip_session_id`, `traveler_group`, `category`, `item_name`, `suggested_quantity`, `priority`, `reason_code`, `explanation`, `is_completed`, `is_user_added`, `revision`, `created_at`, and `updated_at`. `Cultural_guides` stores `cultural_guide_id`, `trip_session_id`, `destination_insight_id`, `category`, `guidance_text`, `source_label`, `review_date`, `confidence_level`, and `created_at`. `Travel_advisories` stores `travel_advisory_id`, `trip_session_id`, `destination_insight_id`, `severity`, `title`, `advisory_text`, `source_label`, `valid_from`, `valid_until`, `review_date`, `verification_url`, and `created_at`.

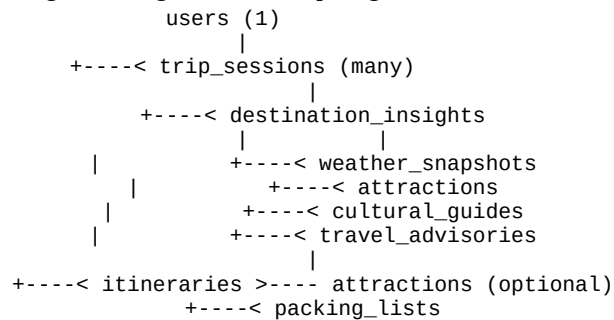
Table 3.10 Shared Field Conventions



Field Group	Design Decision
Coordinates	DECIMAL values with consistent precision
Time	Destination-local date or time plus documented time zone
Status	Controlled ENUM or validated VARCHAR values
External fragments	Bounded JSON with provider and retrieval timestamp
Explanations	Reason code plus readable text

3.6.2 Relationship Structure

Figure 3.5 Logical Relationship Diagram



- One user can own many trip sessions, while each trip session belongs to one user.
- One trip session can compare several destination insights, but only one may be marked selected for a revision.
- A destination insight can have several climate snapshots when comparison windows or providers differ.
- Attractions belong to the trip-specific destination insight so source freshness is preserved.



- An itinerary row belongs to one trip and may reference one attraction; rest and preparation blocks may have no attraction reference.
- Packing items belong directly to the trip because they combine destination, climate, purpose, duration, and traveler context.
- Cultural and safety records may be copied into the trip context with the original source and review date.

Deletion behavior is conservative. Removing a user may cascade to owned trip sessions only after explicit confirmation. Removing a draft trip can cascade to generated children. A referenced attraction uses restricted or controlled deletion so that an existing itinerary is not silently corrupted. Updating provider information creates a new revision rather than rewriting a plan that the user has already reviewed.

3.6.3 Why Relational Storage Is Appropriate

Relational storage is appropriate because PackSwift contains clear ownership, one-to-many relationships, controlled status transitions, and records that must remain mutually consistent. A trip session should not refer to another user's destination insight, and an itinerary should not reference an attraction from an unrelated destination. Foreign keys and transactional operations enforce these rules more reliably than application convention alone.

MySQL also supports the mixed shape of the data. Core fields such as dates, scores, country codes, coordinates, categories, and timestamps are relational and indexed. Limited provider fragments and explanation components can use JSON where their internal fields vary, while frequently queried values are extracted into normal columns. This hybrid use of relational and JSON data preserves query performance without discarding source evidence.

Table 3.11 Data Integrity Controls



Integrity Concern	Control
Cross-user access	Filter every trip query by authenticated user_id
Invalid date range	Service validation and database constraint
Duplicate generation	Composite unique key and idempotency key
Orphaned itinerary block	Foreign key with controlled delete behavior
Stale provider result	retrieved_at, review_date, and confidence level
Unbounded payload growth	Selected fields and size limits for JSON

Repository functions use parameterized SQL and return domain objects rather than raw rows. Database credentials are supplied through environment configuration and are never sent to the browser. Migration files create tables, indexes, constraints, and seed configuration in a repeatable order. A small set of anonymized fixtures supports local development and automated tests.

3.7 Datasets and External Resources

3.7.1 Open-Meteo

Open-Meteo supplies historical weather and climate data. Requests use destination coordinates, date ranges, and selected daily variables such as minimum temperature, maximum temperature, mean temperature, and precipitation. The Historical Weather API provides long-term reanalysis data, while the Climate API provides model-based daily climate values. The report treats these data as statistical evidence and records the selected model and observation period (Open-Meteo, n.d.-a, n.d.-b).

3.7.2 Amadeus APIs

Amadeus supplies destination experiences, attractions, and points of interest. The adapter requests results by location or destination code, maps provider categories into PackSwift categories, preserves provider identifiers and source links, and discards fields not required by the planning modules. Amadeus documentation describes destination experience resources for finding places and activities by geographic location (Amadeus for Developers, n.d.).

3.7.3 REST Countries API

REST Countries supplies normalized country facts including country names, ISO codes, capitals, currencies, languages, population, region, subregion, coordinates, and time zones. The adapter requests or retains only the fields used by PackSwift and maps them to an internal country object. Country facts support destination intelligence and do not replace city-level cultural knowledge (REST Countries, n.d.).



Table 3.12 External Resource Responsibilities

Provider	PackSwift Use	Persisted Metadata
Open-Meteo	Historical and climate statistics	Coordinates, date range, variables, model, retrieval time
Amadeus	Attractions and destination experiences	Provider ID, coordinates, category, source link, retrieval time
REST Countries	Country reference facts	ISO code, selected fields, retrieval time

3.7.4 Data Quality and Resilience

External data is treated as evidence with known limitations, not as unquestionable truth. Each adapter validates HTTP status, content type, required fields, value ranges, and response size. Provider-specific responses are converted into versioned internal objects before they enter business logic. This prevents an external naming change from directly altering the engines.

- Use explicit request timeouts and bounded retries for temporary network failures.
- Respect provider rate limits and avoid duplicate requests within one planning session.
- Cache destination facts and attraction snapshots for a documented freshness period.
- Keep climate calculations reproducible by saving variables, period, model, and derived values.
- Return partial results with a clear source-status message when one provider is unavailable.
- Never replace a missing fact with invented content.
- Log provider failure category, request identifier, and duration without exposing credentials.

Figure 3.6 Provider Resilience Flow



Cultural and safety records require a review date and source label. Expired high-severity notes are not silently shown as current; they are hidden or marked for review according to policy. The interface displays when a plan is based on cached or incomplete data and provides a regenerate action after service recovery.

API credentials remain on the server. Environment-specific configuration separates local, test, and deployment values. Test fixtures simulate normal responses, empty results, invalid values, timeouts, rate-limit responses, and provider schema changes so resilience is evaluated before frontend integration.

3.8 Development Planning

Implementation follows a backend-first roadmap of nine phases. A phase is accepted only when its deliverable is demonstrable and its verification criteria pass. Later phases depend on stable contracts from earlier phases, but corrective iteration is expected when testing reveals an invalid assumption.

3.8.1 Phase One: Database Design

Technical tasks: define the relational foundation for every planning module.

- Translate each project objective into persistent entities and required fields.
- Create data definitions for users, trip_sessions, destination_insights, weather_snapshots, attractions, itineraries, packing_lists, cultural_guides, and travel_advisories.
- Select primary keys, foreign keys, data types, nullable fields, controlled status values, timestamps, and revision fields.
- Define cascade, restrict, and update behavior for each relationship.
- Add unique constraints for email, provider place identity, and idempotent generation.
- Add indexes for recent user trips, ranked destination insights, attraction lookup, itinerary sequence, and active advisories.
- Create repeatable migration files and a small anonymized seed set.
- Write validation fixtures for valid, boundary, and invalid records.
- Document every field in a concise data dictionary.



The phase begins with conceptual relationships and ends with executable SQL migrations. No browser work is required. Sample queries must demonstrate that a user's trip can be retrieved with its selected destination, climate snapshot, attraction set, itinerary blocks, cultural notes, safety notes, and packing items.

Table 3.13 Phase One Acceptance Checks

Acceptance Check	Expected Result
Fresh migration	All nine tables and indexes are created without manual edits
Invalid date range	Record is rejected by validation or constraint
Cross-trip attraction reference	Relationship is rejected
Duplicate generated destination	Composite uniqueness prevents duplication

Deliverable: a versioned MySQL schema containing the nine required tables, migration and rollback scripts, indexes, constraints, a data dictionary, and verified sample records.

Technical justification: the planning engines depend on stable ownership and relationships. Designing persistence first prevents later modules from inventing incompatible representations for the same trip, destination, or source snapshot. It also makes auditability and data isolation structural properties rather than late corrections.

Figure 3.7 Recommended Migration Order

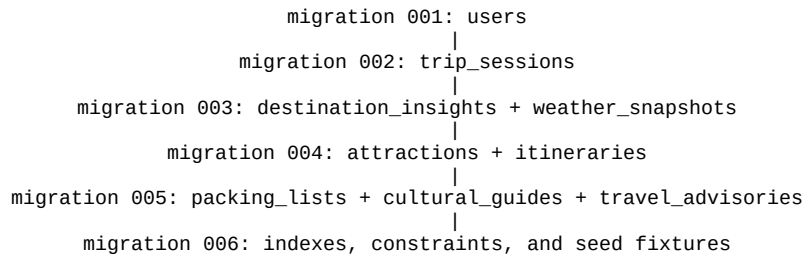


Table 3.14 Phase One Review Evidence



Database Review Question	Required Evidence
Can every generated result be linked to one trip?	Foreign key path and sample query
Can a result be reproduced?	Rule version, source, retrieval time, and input revision
Can stale information be identified?	Review or retrieval timestamp and confidence
Can one user read another user's trip?	Repository filter and authorization test demonstrate denial
Can a generation request be repeated safely?	Unique key or idempotency evidence

Phase One is complete only when the schema can be created from an empty database, exercised with fixtures, rolled back in a test environment, and recreated with identical structure. The accepted data dictionary becomes the naming authority for subsequent server and interface work.

3.8.2 Phase Two: Node.js Backend Setup

Technical tasks: create a maintainable server foundation and connect it to MySQL.

- Initialize the Node.js project with documented scripts for development, testing, linting, and startup.
- Install Express.js, the selected MySQL driver, environment configuration, validation, and test dependencies.
- Create folders for configuration, routes, controllers, services, repositories, provider adapters, middleware, validators, and tests.
- Implement environment validation that fails clearly when required settings are absent.
- Create the Express application, JSON size limit, request identifier, CORS policy, health endpoint, and graceful shutdown.
- Create a pooled MySQL connection and a repository smoke test.
- Implement centralized not-found and error middleware with safe client messages.
- Add a consistent response envelope containing data, source status, warnings, and request identifier.
- Create authentication placeholders or session ownership middleware appropriate to the prototype.
- Write route and repository tests that run independently of the browser.

The server initially exposes only system health, database health, and a minimal trip-session route. This limited vertical slice proves that configuration, routing, validation, service delegation, repository access, error handling, and test execution operate together before external providers or planning rules are introduced.

Table 3.15 Initial Backend Endpoints



Endpoint	Purpose	Expected Status
GET /api/v1/health	Server readiness	200 when application is ready
GET /api/v1/health/db	Database connectivity	200 or explicit degraded state
POST /api/v1/trips	Validated trip-session creation	201 for valid input

Deliverable: a running and tested Node.js and Express.js backend with validated configuration, modular routing, centralized errors, a MySQL connection pool, repository access, and a working trip-session endpoint.

Technical justification: a stable application shell separates infrastructure concerns from travel reasoning. Provider adapters and engines can then be added behind known interfaces, while consistent errors and response envelopes prevent each module from creating a different client contract.

Figure 3.8 Backend Request Path

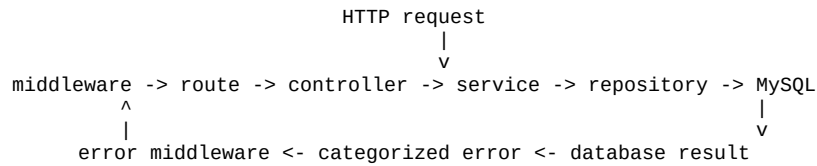


Table 3.16 Backend Component Boundaries

Component	Boundary
Route	Matches method and path; attaches validators
Controller	Translates HTTP input and output only
Service	Applies use-case logic and coordinates dependencies
Repository	Executes parameterized SQL and maps rows
Middleware	Handles shared concerns such as identity, logging, and errors

Acceptance requires automated tests for valid creation, missing fields, invalid dates, unsupported currency codes, excessive request size, database unavailability, and unknown routes. Error responses must avoid stack traces and credentials. Logs must retain enough context for diagnosis through a request identifier.

The folder structure and response conventions are documented for future contributors. This prevents later phases from placing provider calls directly inside routes or embedding SQL inside planning rules.

3.8.3 Phase Three: API Integration

Technical tasks: integrate the three approved providers through isolated adapters.

- Create a shared HTTP client with timeout, request identifiers, safe headers, and bounded retry behavior.
- Implement an Open-Meteo adapter for historical and climate variables required by the climate engine.
- Implement an Amadeus authentication flow and destination experience or point-of-interest requests.
- Implement a REST Countries adapter for country facts and selected response fields.
- Map each provider response into an internal object with stable names and types.
- Validate coordinates, dates, country codes, arrays, numeric ranges, and optional fields.
- Persist retrieval time, source identity, and selected provider fragments.
- Add in-memory or database-backed freshness checks that prevent unnecessary repeated calls.
- Create fixtures for success, empty response, timeout, rate limit, malformed response, and unavailable service.
- Expose internal diagnostic routes only in the development environment.

Adapters return normalized data or categorized errors; they do not decide destination suitability or packing content. This is a critical separation. A change in an Amadeus category or a REST Countries field should require a mapping update, not a rewrite of the destination engine.

Table 3.17 Normalized Provider Minimums



Adapter Output	Required Minimum Fields
Climate series	date, temperature measures, precipitation, source model
Attraction	provider ID, name, category, latitude, longitude
Country facts	ISO code, name, currency, languages, region

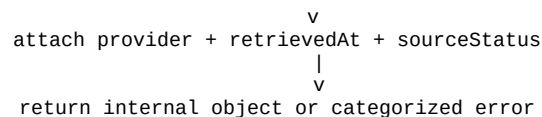
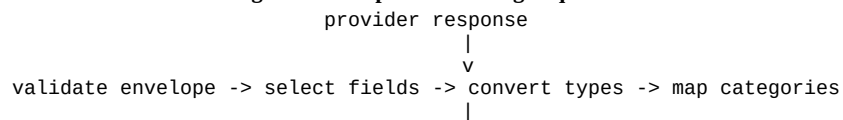
Deliverable: three tested provider adapters that return normalized climate, attraction, destination experience, and country information with caching, provenance, and explicit failure states.

Technical justification: isolating external formats protects the application from provider-specific change and makes the planning engines testable with local fixtures. Provenance and retrieval timestamps are essential because the system synthesizes information from sources with different coverage and freshness.

Table 3.18 Example Normalization Rules

Provider Field	Internal Field	Normalization Rule
country or ISO value	countryCode	Uppercase validated ISO code
lat / lng variants	latitude / longitude	Finite decimal within geographic range
category label	category	Map to controlled PackSwift category
date-time string	retrievedAt	ISO 8601 in UTC
optional rating	providerRelevance	Nullable normalized value from 0 to 100

Figure 3.9 Adapter Processing Sequence



Integration tests use recorded fixtures and do not depend on a live provider for every test run. A small separate smoke suite can verify credentials and current connectivity when explicitly enabled. The accepted phase must demonstrate that one unavailable provider produces a partial planning response rather than an unhandled server failure.

3.8.4 Phase Four: Climate Analysis Module

Technical tasks: convert historical and climate values into cautious, explainable seasonal guidance.

- Define the seasonal comparison window and configurable historical period.
- Request daily minimum, maximum, and mean temperature plus precipitation variables.
- Remove invalid or missing observations and record the remaining sample size.
- Calculate temperature means, ranges, percentile-based extremes, mean precipitation, and rainy-day ratio.
- Classify cool, mild, hot, dry, frequently wet, and variable conditions using versioned thresholds.
- Generate a summary that states likely conditions, variation, observation period, and uncertainty.
- Persist raw selected values, derived statistics, category labels, confidence, and rule version.
- Return climate signals for destination, attraction, itinerary, preparation, and packing logic.
- Create unit tests for tropical wet, cool variable, hot dry, sparse data, and boundary cases.

The module distinguishes data preparation, calculation, classification, and wording. A numerical function should not create user-facing prose, and a wording function should not silently change the numeric category. This separation supports independent validation by the supervisor.

Table 3.19 Climate Module Fixtures



Test Fixture	Expected Classification
High temperature; low rain ratio	Hot and predominantly dry
Moderate temperature; high rain ratio	Mild and frequently wet
Large year-to-year spread	Variable with reduced confidence
Insufficient valid days	Unavailable or low confidence

Deliverable: a tested Climate Analysis Engine that produces persisted temperature and rainfall statistics, climate categories, uncertainty statements, and reusable context signals for other modules.

Technical justification: climate affects destination suitability, outdoor activity relevance, daily pacing, preparation, and packing. Implementing the module before destination and guide logic provides one consistent interpretation instead of several conflicting calculations across the server.

Table 3.20 Climate Verification Criteria

Metric	Verification Method	Pass Condition
Mean temperature	Independent fixture calculation	Matches expected value within rounding rule
Rainy-day ratio	Known count of wet days	Exact ratio before display rounding
Confidence	Vary valid observation count	Drops at documented threshold
Determinism	Run same input twice	Normalized outputs are identical
Uncertainty wording	Inspect boundary fixtures	No certainty language for variable data

Figure 3.10 Climate Transformation

historical values
| clean and validate
v
descriptive statistics
| apply versioned thresholds



v
climate categories
| attach confidence and period
v
plain-language seasonal guidance

A supervisor review should compare several generated summaries with the underlying data table. The interface contract must include the calculation period and confidence, ensuring that later frontend work cannot display a climate category without its qualifying context.

3.8.5 Phase Five: Destination Intelligence Module

Technical tasks: rank supported destinations and assemble concise destination intelligence.

- Define the supported candidate set and maintained affordability bands.
- Validate budget currency, travel duration, preferred region, interests, purpose, and climate tolerance.
- Join candidate destinations with country facts, climate signals, and available attraction coverage.
- Calculate budget, preference, climate, duration, and seasonal component scores.
- Apply user hard constraints before weighted ranking.
- Generate strengths, cautions, assumptions, and confidence for each candidate.
- Allow a known destination to be analyzed without forcing comparison.
- Persist ranked destination_insights and mark one selected insight per revision.
- Create routes to generate, list, select, and regenerate destination analysis.
- Test score boundaries, ties, missing coverage, currency mismatch, and excluded regions.

Destination intelligence also presents capital, currency, languages, population, region, time-zone context, and climate summary in a consistent layout. Population is contextual information and is not used as a quality score. The module must avoid implying that a larger or more popular destination is automatically more appropriate.

Table 3.21 Destination Score Interpretation



Score Range	Interpretation
80-100	Strong fit under the recorded assumptions
65-79	Good fit with visible cautions
50-64	Possible fit; review trade-offs
Below 50	Weak fit for the current constraints

Deliverable: a Destination Analysis Engine and destination intelligence service that return ranked or evaluated destinations with component scores, country facts, climate context, assumptions, and readable reasons.

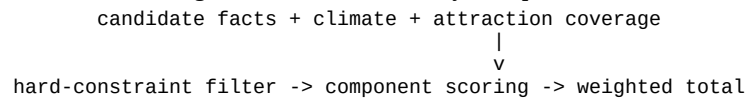
Technical justification: destination selection is the organizing decision for the remaining plan. Implementing it after provider and climate normalization ensures that scoring is based on consistent data. Component scores make the outcome inspectable and allow evaluation beyond whether the user simply accepts the top result.

Table 3.22 Illustrative Destination Score Components

Candidate	B	P	C	D	S	Total
A	85	90	60	80	70	79.25
B	95	65	75	70	75	79.00
C	60	95	90	85	80	80.50

The example shows why the explanation matters: Candidate C ranks first despite weaker budget fit because it has stronger preference and climate fit. The system must still display the cost caution. A user may select Candidate A because budget predictability is more important, then regenerate later modules for that choice.

Figure 3.11 Destination Analysis Sequence



Acceptance includes an explanation-coverage test and a supervisor review of at least five contrasting trip scenarios. Score weights and affordability assumptions are documented and versioned so later calibration does not erase the basis of earlier results.

3.8.6 Phase Six: Packing Recommendation Engine

Technical tasks: generate editable, traveler-aware packing lists from shared trip context.

- Define a controlled packing taxonomy for documents, clothing, climate protection, personal care, electronics, health reminders, child needs, and pet needs.
- Create universal essentials and optional men, women, children, and pet rule sets.
- Define temperature, rainfall, duration, destination, cultural, and purpose triggers.
- Implement bounded quantity rules and long-stay laundry reminders.
- Merge overlapping results and preserve the highest priority and all relevant reasons.
- Generate separate traveler-group views while allowing the user to move or copy an item.
- Persist priority, quantity, explanation, completion, user-added status, and revision.
- Create routes to generate, retrieve, edit, complete, delete, restore, and regenerate items.
- Test zero-count traveler groups, long trips, climate boundaries, duplicate rules, and user edits.

The engine avoids prescriptive gender assumptions. Men and women lists begin with the same universal base and expose optional personal or clothing items that can be removed. Child logic is age-aware only when the user supplies an age band. Pet logic requests species and relevant travel context before suggesting specialized items.

Table 3.23 Packing Priority Levels



Priority	Meaning
Essential	Likely required for the trip context or traveler safety
Recommended	Strongly useful under the recorded climate or purpose
Optional	Potentially useful; depends on personal preference

Deliverable: a tested Packing Recommendation Engine with configurable rules, grouped editable lists, bounded quantities, duplicate resolution, reason codes, and persistent completion state.

Technical justification: packing is most useful after destination and climate context are stable. Building the engine at this point allows one item to reflect several factors without duplicating climate calculations or country logic. Editable results preserve personal control and reduce the risk of overgeneralized guidance.

Figure 3.12 Packing Generation Flow

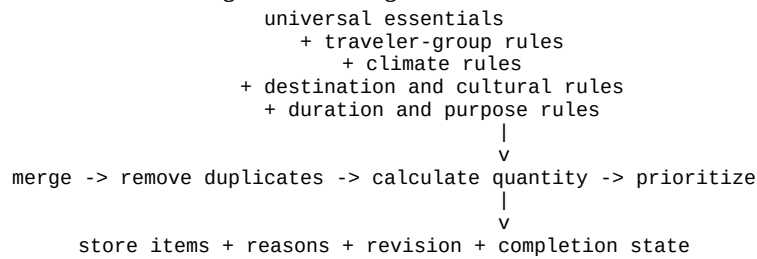


Table 3.24 Packing Evaluation Scenarios



Scenario	Key Expected Items or Notes
Five-day wet city trip	Rain protection, water-resistant storage, indoor alternative note
Ten-day business trip	Formal option, document organizer, laundry reminder
Family trip with child	Child comfort, identification, age-adjusted essentials
Trip with pet	Carrier, feeding, comfort, record verification reminders

Evaluation checks precision through expert review of irrelevant suggestions, recall through required-item fixtures, duplicate rate, explanation coverage, and edit persistence. The system never presents the generated list as exhaustive; the interface asks the traveler to review personal, medical, and legal requirements.

3.8.7 Phase Seven: Itinerary Generator

Technical tasks: convert ranked attractions and trip constraints into an ordered multi-day plan.

- Normalize attraction category, coordinates, estimated visit time, opening note, and relevance components.
- Define daily capacity from pace level and traveler composition.
- Group attractions by geographic proximity and compatible category.
- Seed each day with one high-priority attraction before filling remaining capacity.
- Adjust indoor and outdoor suitability using climate categories.
- Insert meal, rest, transport, climate, and verification notes.
- Avoid duplicate attractions and excessive repetition of one category.
- Store each plan block with day, sequence, time block, attraction reference, and reason.
- Create routes to generate, reorder, remove, add, and regenerate itinerary blocks.
- Test one-day, multi-day, sparse-attraction, wet-weather, family-pace, and missing-hour cases.

The first version uses rule-based sequencing, not an advanced route optimizer. Geographic clustering can use approximate coordinate distance for grouping, while the interface clearly labels travel-time estimates as approximate. This boundary keeps the implementation feasible and prevents false precision.

Table 3.25 Illustrative Daily Capacity Rules



Pace	Major Blocks per Day	Default Rest Buffer
Relaxed	2	Generous
Balanced	3	Standard
Intensive	4	Minimum retained buffer

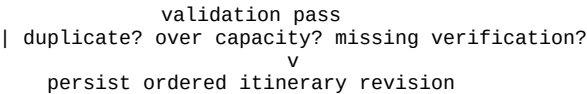
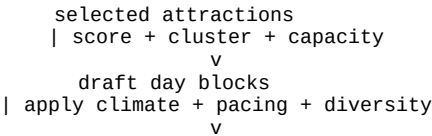
Deliverable: a tested Itinerary Generator that creates editable day and time blocks, groups compatible attractions, applies climate and pacing rules, records reasons, and persists sequence revisions.

Technical justification: an itinerary should be generated only after attraction relevance, climate context, destination selection, and trip duration are established. This dependency order ensures that sequencing is based on shared facts and that a changed destination can invalidate and regenerate the correct plan records.

Table 3.26 Illustrative Itinerary Output

Day	Time Block	Plan Block	Reason
1	Morning	High-priority cultural attraction	Strong interest match; indoor
1	Afternoon	Nearby local history attraction	Same geographic cluster
1	Evening	Flexible food district suggestion	Purpose and preference fit
2	Morning	Outdoor landmark with weather check	High relevance; verify conditions

Figure 3.13 Itinerary Validation Flow



Evaluation measures attraction coverage, duplicate count, daily capacity violations, category diversity, weather-note accuracy, and user-rated coherence. A supervisor should inspect generated plans for contrasting destinations and confirm that missing operating information produces a verification note rather than fabricated detail.

3.8.8 Phase Eight: Frontend Development

Technical tasks: connect the stable backend services to a minimalist and accessible browser experience.

- Create semantic HTML for trip input, destination comparison, climate summary, destination guide, itinerary, and packing views.
- Create a simple progress model that shows the current stage and allows revision of completed stages.
- Implement responsive CSS with readable line length, clear focus, sufficient contrast, and restrained visual hierarchy.
- Use Vanilla JavaScript modules for state, validation, API requests, rendering, and accessible status messages.
- Display score components, explanations, source timestamps, confidence, and partial-data warnings.
- Allow itinerary reordering and packing edits through keyboard-accessible controls as well as pointer interaction.
- Create loading, empty, unavailable, stale-data, validation, and retry states.
- Test keyboard navigation, focus order, labels, error association, target size, responsive layout, and reduced motion.

Deliverable: a responsive HTML, CSS, and Vanilla JavaScript interface connected to all planning endpoints and capable of creating, reviewing, editing, and regenerating a complete trip plan.

Technical justification: frontend work follows backend stabilization so the interface is built against verified response contracts. Progressive disclosure reduces visual and cognitive load, while explanations and source status preserve the transparency established in the service layer.

Table 3.27 Frontend Screen Structure



Screen	Primary Content	Main Action
Trip setup	Budget, dates, purpose, travelers, preferences	Analyze trip
Destination	Ranked candidates and score components	Select destination
Climate and guide	Likely conditions, facts, attractions, notes	Generate plan
Itinerary	Editable day and time blocks	Save revision
Packing	Grouped editable items and reasons	Mark and export

3.8.9 Phase Nine: Testing and Evaluation

Technical tasks: verify technical correctness and evaluate whether the assistant reduces planning effort while producing useful, explainable guidance.

- Run unit tests for scoring, climate statistics, quantity rules, duplicate handling, and itinerary capacity.
- Run integration tests for routes, repositories, provider adapters, persistence, and partial failures.
- Run system scenarios from trip creation through final plan revision.
- Evaluate security controls for authorization, validation, parameterized SQL, secret handling, and safe errors.
- Conduct accessibility checks and keyboard-only task completion against WCAG 2.2 considerations.
- Recruit representative users for task-based usability evaluation and record completion time, errors, assistance, and satisfaction.
- Ask domain reviewers to rate destination relevance, climate wording, itinerary coherence, cultural caution, safety labeling, and packing usefulness.
- Document limitations, failed cases, corrections, and final acceptance evidence.

Deliverable: a complete test suite, evaluation protocol, anonymized results, defect log, corrected release candidate, and traceability matrix linking objectives to evidence.

Technical justification: the system cannot be evaluated only by whether pages load. Its academic value depends on correctness, relevance, transparency, resilience, usability, and responsible uncertainty communication. Multiple methods are required because no single metric captures all of these qualities.

Table 3.28 Evaluation Framework



Evaluation Dimension	Indicative Measure
Correctness	Expected rule outputs and database integrity
Relevance	Reviewer rating of recommendations
Explainability	Reason coverage and user understanding
Usability	Task completion, time, errors, and SUS score
Resilience	Controlled behavior during provider failure
Performance	Median and high-percentile response time

3.9 Summary of Chapter Three

This chapter presented a complete methodology for PackSwift as an AI-powered personal travel planning assistant. The design uses a layered architecture in which a minimalist browser interface communicates with Express.js routes, the routes delegate to a Travel Planning Service Layer, MySQL preserves trip and source records, and isolated adapters integrate Open-Meteo, Amadeus, and REST Countries.

The intelligence is deterministic and explainable. The Destination Analysis Engine evaluates budget, dates, destination, duration, purpose, and preferences. The Climate Analysis Engine converts historical and climate data into temperature, rainfall, variability, and confidence summaries. The Travel Guide Generator assembles attractions, preparation tasks, cultural notes, and time-stamped safety information. The Packing Recommendation Engine creates editable lists for men, women, children, and pets. The Itinerary Generator converts ranked attractions into climate-aware and pace-aware daily blocks.

The database design centers on nine tables: users, trip_sessions, destination_insights, weather_snapshots, attractions, itineraries, packing_lists, cultural_guides, and travel_advisories. Foreign keys, indexes, timestamps, revisions, reason codes, and provider metadata support integrity and traceability. External information is normalized and labeled with source status so missing or stale data is never silently replaced with invented content.

The backend-first roadmap contains nine phases: Database Design, Node.js Backend Setup, API Integration, Climate Analysis Module, Destination Intelligence Module, Packing Recommendation Engine, Itinerary Generator, Frontend Development, and



Testing and Evaluation. Each phase defines technical tasks, a deliverable, and a technical justification. The sequence provides a feasible implementation path while preserving the project's academic objectives, stack constraints, responsible limitations, and minimalist user experience.

The resulting methodology aligns all chapters around one concept: when a user decides to travel, PackSwift structures the essential planning decisions in one place. The project does not attempt to replace public authorities or guarantee future conditions; it provides a coherent, editable, source-aware plan that helps the traveler know where to go, what to expect, what to prepare, and what to pack.

References

- Abbasi-Moud, Z., Hosseinabadi, S., Kelarestaghi, M., & Eshghi, F. (2022). CAFOB: Context-aware fuzzy-ontology-based tourism recommendation system. *Expert Systems with Applications*, 199, 116877. <https://doi.org/10.1016/j.eswa.2022.116877>
- Amadeus for Developers. (n.d.). Destination experiences APIs tutorial. Retrieved July 15, 2026, from <https://developers.amadeus.com/self-service/apis-docs/guides/developer-guides/resources/destination-experiences/>
- Borràs, J., Moreno, A., & Valls, A. (2014). Intelligent tourism recommender systems: A survey. *Expert Systems with Applications*, 41(16), 7370-7389. <https://doi.org/10.1016/j.eswa.2014.06.007>
- Braunhofer, M., Elahi, M., Ge, M., Ricci, F., & Schievenin, T. (2013). STS: Design of weather-aware mobile recommender systems in tourism. In *Proceedings of the First International Workshop on Intelligent User Interfaces: Artificial Intelligence Meets Human-Computer Interaction* (CEUR Workshop Proceedings, Vol. 1125). <https://ceur-ws.org/Vol-1125/paper5.pdf>
- De Choudhury, M., Feldman, M., Amer-Yahia, S., Golbandi, N., Lempel, R., & Yu, C. (2010). Automatic construction of travel itineraries using social breadcrumbs. In *Proceedings of the 21st ACM Conference on Hypertext and Hypermedia* (pp. 35-44). Association for Computing Machinery. <https://doi.org/10.1145/1810617.1810626>
- Express.js. (n.d.-a). Routing. Retrieved July 15, 2026, from <https://expressjs.com/en/guide/routing/>
- Express.js. (n.d.-b). Using middleware. Retrieved July 15, 2026, from <https://expressjs.com/en/5x/guide/using-middleware/>
- Felfernig, A., Gordea, S., Jannach, D., Teppan, E., & Zanker, M. (2006). A short survey of recommendation technologies in travel and tourism. *ÖGAI Journal*, 25(4), 17-22.
- Gavalas, D., Konstantopoulos, C., Mastakas, K., & Pantziou, G. (2014). Mobile recommender systems in tourism. *Journal of Network and Computer Applications*, 39, 319-333. <https://doi.org/10.1016/j.jnca.2013.04.006>



- Gretzel, U., Sigala, M., Xiang, Z., & Koo, C. (2015). Smart tourism: Foundations and developments. *Electronic Markets*, 25(3), 179-188. <https://doi.org/10.1007/s12525-015-0196-8>
- Kabassi, K. (2010). Personalizing recommendations for tourists. *Telematics and Informatics*, 27(1), 51-66. <https://doi.org/10.1016/j.tele.2009.05.003>
- Kurashima, T., Iwata, T., Irie, G., & Fujimura, K. (2010). Travel route recommendation using geotags in photo sharing sites. In *Proceedings of the 19th ACM International Conference on Information and Knowledge Management* (pp. 579-588). Association for Computing Machinery. <https://doi.org/10.1145/1871437.1871513>
- Liu, Q., Ge, Y., Li, Z., Chen, E., & Xiong, H. (2011). Personalized travel package recommendation. In *2011 IEEE 11th International Conference on Data Mining* (pp. 407-416). IEEE. <https://doi.org/10.1109/ICDM.2011.118>
- Namahoot, C. S., Brückner, M., & Panawong, N. (2015). Context-aware tourism recommender system using temporal ontology and Naive Bayes. In H. Unger, P. Meesad, & S. Boonkroong (Eds.), *Recent advances in information and communication technology 2015* (pp. 183-194). Springer. https://doi.org/10.1007/978-3-319-19024-2_19
- Node.js. (n.d.). About Node.js. Retrieved July 15, 2026, from <https://nodejs.org/en/about/>
- Noguera, J. M., Barranco, M. J., Segura, R. J., & Martínez, L. (2012). A mobile 3D-GIS hybrid recommender system for tourism. *Information Sciences*, 215, 37-52. <https://doi.org/10.1016/j.ins.2012.05.010>
- Open-Meteo. (n.d.-a). Climate API. Retrieved July 15, 2026, from <https://open-meteo.com/en/docs/climate-api>
- Open-Meteo. (n.d.-b). Historical Weather API. Retrieved July 15, 2026, from <https://open-meteo.com/en/docs/historical-weather-api>



- Oracle. (n.d.). MySQL 8.4 Reference Manual: Data types. Retrieved July 15, 2026, from <https://dev.mysql.com/doc/refman/8.4/en/data-types.html>
- Ravi, L., & Vairavasundaram, S. (2016). A collaborative location based travel recommendation system through enhanced rating prediction for the group of users. *Computational Intelligence and Neuroscience*, 2016, 1291358. <https://doi.org/10.1155/2016/1291358>
- REST Countries. (n.d.). Countries API. Retrieved July 15, 2026, from <https://restcountries.com/docs/countries>
- Souffriau, W., Vansteenwegen, P., Vertommen, J., Vanden Berghe, G., & Van Oudheusden, D. (2008). A personalized tourist trip design algorithm for mobile tourist guides. *Applied Artificial Intelligence*, 22(10), 964-985. <https://doi.org/10.1080/08839510802379626>
- Vansteenwegen, P., Souffriau, W., Vanden Berghe, G., & Van Oudheusden, D. (2011). The city trip planner: An expert system for tourists. *Expert Systems with Applications*, 38(6), 6540-6546. <https://doi.org/10.1016/j.eswa.2010.11.085>
- World Wide Web Consortium. (2024, December 12). Web Content Accessibility Guidelines (WCAG) 2.2. <https://www.w3.org/TR/WCAG22/>

